

Reflection Journal Report for Lab 05

Prepared for Prof. Vishwantha Rao, HCC ITAI 1371 Machine Learning

Prepared by FA26_ML_6171_16347 3_ITAI1371 Group

19-September-2026

The purpose of this report is for each student to reflect on their learning journey for ITAI 1371 Machine Learning (ML) Lab 05 Exploratory Data Analysis (EDA). This Reflection Journal Report consists of separate parts for each student in the group. Each student in the group was responsible for their own content and report formatting in each of the sections below.

This reflection journal entry is for Lab 05 Data Preparation and was prepared by Irene Ramos.

Understanding Insights & New Breakthroughs

Through this lab, I learned that raw data is rarely ready for machine learning algorithms. The main achievement was to realize just how mathematical the algorithms were – they do not work with text and they do not accept null data (NaN). It was also clear for me that One-Hot Encoding is an important step, as it stops the algorithm from making mathematical assumptions about the relationship between categories, for instance, assigning numerical value importance to ports or locations.

Analyzing the Problem / Problem Solving Strategies

Two major issues in the Titanic dataset were missing values, particularly in the 'Age' feature, and non-numeric categorical variables. In order to address the problem of missing ages, I calculated the median since it is not affected by outliers such as very aged individuals, and then I used `.fillna()`. The other issue was that of categorical variables, where I used `pd.get_dummies()` to obtain binary values for 'Sex' and 'Embarked'. It should be noted that there was an issue while imputing the missing values; specifically, I got a FutureWarning of changing a value in a copy of a DataFrame using `.loc[]` with `inplace=True`.

Real-World Applications

This data preparation process is directly applicable to my work with physical security at Google. In cases where there are huge amounts of data from badge reader logs, turnstiles access data or perimeter sensors, data drops do occur. In cases where a particular door reader is not recording the exit times, I am now in a position to impute the "time-in-building" data using the median method instead of dropping the whole log and losing its context. One-Hot encoding is ideal for discretely classifying physical security incidents by

transforming them to Alarm_Tailgating, Alarm_DoorPropped or Alarm_GlassBreak. Finally, feature scaling is essential for anomaly detection; it ensures that a model is not biased towards high-value data while ignoring low-value important data.

Connecting New & Prior Knowledge

In this case, I was able to see how my prior knowledge of statistics could be applied practically in relation to ML constraints by understanding why the median is far more robust compared to the mean when dealing with a skewed distribution of data. Moreover, this helped me understand how these principles relate to the structure of the algorithms since distance-based algorithms are highly dependent on feature scaling while decision trees do not need it due to using simple thresholding mechanisms.

Emerging Questions

Whereas the computation of the median was a reasonable solution to the issue of missing values in the lab's Age attribute, I am now uncertain about how to impute in cases where the physical security data does not seem to be MAR? For instance, if a camera feed or badge reader stops functioning because of a deliberate power failure, would imputing "median" levels of activity would it cover up a major security problem?

Changing Approach & Critical Thinking

Looking ahead, machine learning will be seen by me less as a stand-alone tool and more as an activity of conscientious data engineering. The warning from the Pandas library regarding the future version of the code was another reminder for me to be very suspicious of any changes that might break our scripts for automated reporting on physical security.

This reflection journal entry is for Lab 05 Data Preparation and was prepared by Yuanyuan Kou :

In this lab, I learned how to prepare raw data before using it in a machine learning model. Before completing the lab, I knew that data cleaning was important, but I did not fully understand why each step was necessary. By working with the Titanic dataset, I was able to see real data problems and solve them step by step.

The first task was handling missing values in the Age column. When I checked the dataset, I found that 177 passengers did not have an age recorded. At first, I wondered why we could not simply delete these rows. Then I realized that deleting them would remove a large amount of useful information. Instead, I calculated the median age, which was 28, and used it to fill in the missing values. This helped me understand the meaning of imputation. I also learned that the median is often better than the mean when the data may contain very high or very low values.

The second task was One-Hot Encoding. The Sex and Embarked columns contained text, but machine learning models usually need numerical data. I used `pd.get_dummies()` to change these categories into new columns with True and False values. For example, the original Sex column became Sex_male. At first, this result looked a little strange to me because I expected to see only 0 and 1. After studying the output, I understood that True and False can also represent 1 and 0. This task helped me understand how a model can use categorical information.

The third task was feature scaling. The values in Age and Fare had very different ranges, so I used `StandardScaler` to put them on a similar scale. When I first saw negative numbers after scaling, I was confused because a passenger could not have a negative age or fare. I then learned that these were not the original values. A negative scaled value only means that the original value was below the average. This was an important new idea for me.

This lab also helped me better understand that different machine learning models have different needs. Feature scaling can be important for models that use distance or are affected by the size of numbers. However, a Decision Tree usually does not need scaling because it makes decisions by splitting the data according to conditions.

Overall, this lab gave me a clearer understanding of data preparation. I learned that raw data is usually not ready to be used directly in a model. Missing values, text categories, and different numerical scales must be handled carefully. I still need more practice remembering the Python syntax, but I now understand the purpose of each step. I feel more comfortable reading the code and checking the results than I did before completing this lab.

This reflection journal entry is for Lab 05 Data Preparation and was prepared by Xiomara Torres:

Through this lab I was able to gain a better understanding of why data preparation is a very important part of machine learning. Working with the Titanic dataset helps me understand how missing values and features with different numerical ranges can affect how a machine learning model processes information.

One of the main problems in this dataset was that the age column had a lot of missing values. To fill in those gaps, we used the code `.fillna()` to replace those missing values with the median age. Another issue we encountered was that some values were written in text, which made it difficult for the model to understand the data. Using the code `pd.get_dummies()` we converted the text in the sex and embarked categories into numerical data that the model could interpret.

I see data preparation being a critical part of machine learning across all industries because data will never be perfect, but to get very close to accurate results it must be treated and curated to be as perfect as possible. Preparing data correctly before training a model helps make that information much more useful and prevent errors.

This lab has helped connect some of the previous concepts we have learned about and tied into python which I am currently learning as well. Functions like `.median()`, `.fillna()` and `pd.get_dummies()` make more sense when they are put into practice and help get a deeper understanding of how they help the machine learning process.

One question that emerged while I worked through this model was how handling missing data in different ways would affect the accuracy of the model. Using median age gave us a certain outcome, but what if we had used mode or mean or removed the data entirely. It would be an interesting test to use all those different methods and figure out which one is the most accurate.

Going forward, I will pay more attention to the data itself before focusing on building or training a machine learning model. Good data on a simple model is way more valuable than messy data on a complex model. This lab allowed me to see that getting a dataset does not mean it is immediately ready to be used. It needs to be corrected and checked for missing information and translated into a language that the model will be able to interpret.

This reflection journal entry is for Lab 05 EDA and was prepared by Eric Roberson:

I. Understanding Insights & New Breakthroughs

The biggest breakthrough in this lab was seeing that data preparation isn't a mechanical checklist — it's a set of judgment calls, and each one has a "why" behind it. The median-over-mean imputation decision, the choice between Label Encoding and One-Hot Encoding, and the decision of whether to scale features at all are all case-by-case assessments of the data's underlying structure. That mirrors something I do constantly in Technical Risk & Controls: deciding how to treat an incomplete control assessment or a partial finding. Just as filling in a missing "Age" value with the median avoids letting one extreme outlier distort the whole model, avoiding an overreaction to one outlier vulnerability finding — while still tracking it — keeps a risk register accurate rather than skewed. The slide deck's MCAR/MAR/MNAR framework (missing completely at random, missing at random, missing not at random) was also a breakthrough for me: it's essentially the same triage logic I apply when a control test comes back with incomplete evidence. Not all "missing data" is equally dangerous — some gaps are noise, and some gaps are a symptom hiding a real problem, which is the MNAR case the deck calls "dangerous, do not drop."

II. Analyzing the Problem / Problem-Solving Strategies

The lab's problem-solving pattern was: identify the defect in the raw data, choose a remediation strategy proportional to the defect, and verify the fix. That's the exact structure of a vulnerability remediation workflow. Missing values are the equivalent of an unpatched or undocumented asset — you can't just ignore it because most models (and most audits) can't process a gap. The lab walked through remediation options ranging in aggressiveness: deletion (the "nuclear option," per the deck, appropriate only when the missing data is a small percentage of rows or the column is mostly empty), simple imputation (mean/median/mode), and advanced imputation (KNN or iterative/regression-based). That's directly analogous to how I'd triage findings in a risk register — some low-severity gaps get closed with a quick compensating control (like mean/median imputation), while a systemic control failure needs a more resource-intensive remediation, similar to KNN or iterative imputation pulling in more context before deciding on a fix. The encoding and scaling tasks reinforced a second problem-solving habit: match the technique to the nature of the data (nominal vs. ordinal, skewed vs. normal) rather than defaulting to one tool for everything — the same discipline required when selecting a remediation approach based on the specific control gap rather than applying a one-size-fits-all fix.

III. Real-World Applications

Outside of the classroom, this lab's techniques map onto core parts of my resiliency and risk work:

- Missing values / imputation → When BC/DR test results, control evidence, or system inventories have gaps, I have to decide whether to exclude the incomplete record, apply a reasonable default (analogous to imputation), or flag it as a standalone risk (analogous to the "indicator feature" concept from the deck, where the fact that something was missing becomes a signal in itself — e.g., "this recovery time objective was never validated" is itself a finding, not just a blank field).
- One-Hot Encoding → Turning a categorical field like an asset's criticality tier or business unit into distinct binary flags is exactly what happens when I convert qualitative risk categories (e.g., "Critical," "High," "Medium," "Low") into structured fields for a risk-scoring model or a dashboard that leadership can filter and sort — text alone doesn't compute a risk score, it needs to be structured first.

- Feature Scaling → Comparing metrics that live on wildly different scales — like number of vulnerabilities (0–500) against dollar impact of an outage (0–\$10M) — without normalizing them first is a classic way to accidentally let the dollar-impact metric dominate a composite risk score, purely because its raw numbers are bigger, not because it's actually more important.
- Decision Trees not needing scaling → This is a good reminder that not every analytical tool requires the same data preparation. Some risk-scoring or prioritization frameworks I use are threshold/rule-based (closer to a decision tree — "is this system internet-facing?" then "is patching overdue by 30+ days?"), and those don't care about raw scale the way a weighted, distance-based scoring model would.

IV. Connecting New & Prior Knowledge

This lab built directly on the EDA (exploratory data analysis) module — the deck frames data prep as "the bridge between raw data and model-ready data," picking up exactly where EDA left off by fixing the problems EDA surfaced. That progression parallels the assessment-to-remediation lifecycle I run in Technical Risk & Controls: first you assess/discover (EDA — find the missing values, find the miscategorized fields, find the scale mismatches), then you remediate (data prep — impute, encode, scale) before the "model" (in my world, the control environment or the resiliency posture) can be trusted to produce reliable output. I also connected the lab's "Golden Rule" for scaling — fit your scaler on training data only, never on test data, to prevent leakage — to a principle I already apply in control testing: you validate a control using data or a scenario that wasn't used to design the control in the first place, or you get an artificially clean result that doesn't hold up under real conditions. Same failure mode, different domain.

V. Emerging Questions

- When a data gap in a risk or resiliency dataset is closed with an assumption (analogous to mean/median imputation), how much of that assumption should be disclosed downstream — should every imputed or estimated data point carry a visible "flag" (like the deck's Age_Missing indicator column) so reviewers and auditors know which numbers are observed versus inferred?
- The deck's warning about One-Hot Encoding's "curse of dimensionality" (100 cities → 100 columns) raises a question for risk scoring specifically: at what point does adding more granular categorical detail into a model (e.g., every individual application vs. grouping into "Top 10 systems + Other") stop improving accuracy and start adding noise or overfitting risk to a resiliency model?
- Feature engineering (creating FamilySize or Title from raw Titanic fields) suggests real predictive power often comes from combining existing fields rather than collecting new ones. What existing fields in a typical risk/control dataset (e.g., days since last patch × criticality tier, or number of open findings × business impact) could be combined into a more predictive "composite risk feature," the way Speed² captured the true physics of braking distance in the polynomial example?

VI. Changing Approach & Critical Thinking

Lab 05 asked us to respond to the following critical thinking questions in markdown:

1. Why median over mean for imputation?

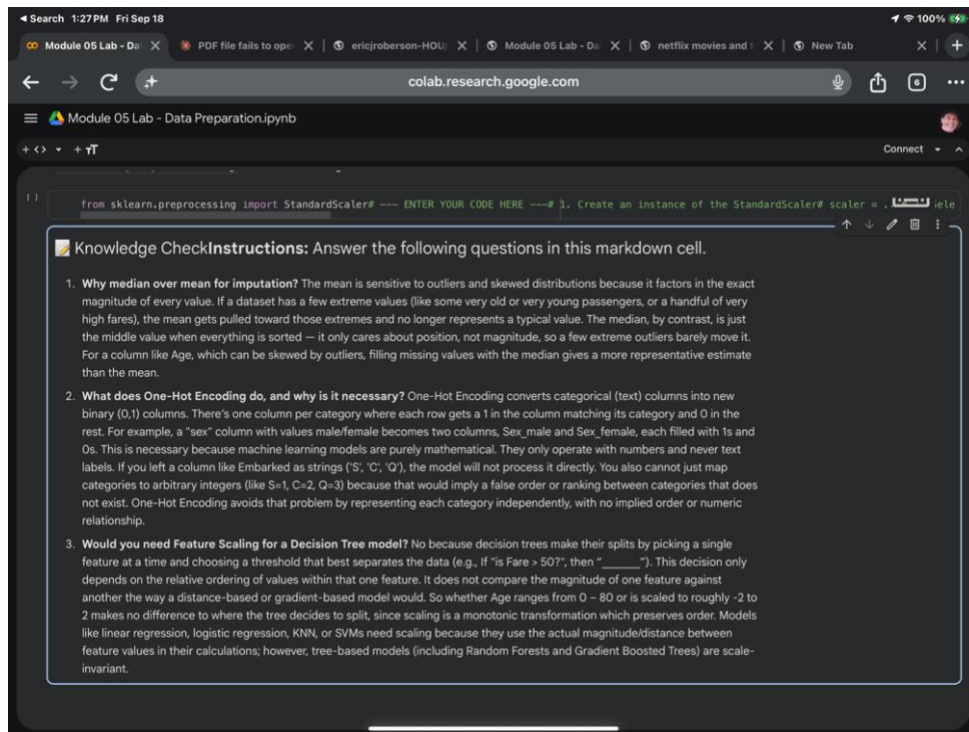
The mean is sensitive to outliers and skewed distributions because it factors in the exact magnitude of every value. If a dataset has a few extreme values (like some very old or very young passengers, or a handful of very high fares), the mean gets pulled toward those extremes and no longer represents a typical value. The median, by contrast, is just the middle value when everything is sorted — it only cares about position, not magnitude, so a few extreme outliers barely move it. For a column like Age, which can be skewed by outliers, filling missing values with the median gives a more representative estimate than the mean.

2. What does One-Hot Encoding do, and why is it necessary?

One-Hot Encoding converts categorical (text) columns into new binary (0,1) columns. There's one column per category where each row gets a 1 in the column matching its category and 0 in the rest. For example, a "sex" column with values male/female becomes two columns, Sex_male and Sex_female, each filled with 1s and 0s. This is necessary because machine learning models are purely mathematical. They only operate with numbers and never text labels. If you left a column like Embarked as strings ('S', 'C', 'Q'), the model will not process it directly. You also cannot just map categories to arbitrary integers (like S=1, C=2, Q=3) because that would imply a false order or ranking between categories that does not exist. One-Hot Encoding avoids that problem by representing each category independently, with no implied order or numeric relationship.

3. Would you need Feature Scaling for a Decision Tree model?

No, because decision trees make their splits by picking a single feature at a time and choosing a threshold that best separates the data (e.g., If "is Fare > 50?", then "_____"). This decision only depends on the relative ordering of values within that one feature. It does not compare the magnitude of one feature against another the way a distance-based or gradient-based model would. So whether Age ranges from 0 – 80 or is scaled to roughly -2 to 2 makes no difference to where the tree decides to split, since scaling is a monotonic transformation which preserves order. Models like linear regression, logistic regression, KNN, or SVMs need scaling because they use the actual magnitude/distance between feature values in their calculations; however, tree-based models (including Random Forests and Gradient Boosted Trees) are scale-invariant.



Reflection Journal Report for Lab 05

ITAI 1371 Machine Learning — Lab 05: Data Preparation

Prepared by Carmen ESONO

19-September-2026

The purpose of this reflection is to describe what I learned during Lab 05 on Data Preparation. This lab helped me understand why data must be cleaned and prepared before it can be used in a machine learning model. I also learned how different data preparation techniques can change the way a machine learning algorithm works.

Understanding Insights & New Breakthroughs

Through this lab, I learned that raw data is usually not ready to be used directly by a machine learning algorithm. One important thing I understood was how missing values, text categories, and different numerical ranges can cause problems for a model. I learned that missing values can be replaced with a useful value, such as the median, instead of deleting the entire row. I also learned that One-Hot Encoding changes categorical information into numerical values that a model can understand. These steps helped me see that data preparation is an important part of machine learning, not just a small step before training a model.

Analyzing the Problem / Problem Solving Strategies

While working with the Titanic dataset, I noticed that the Age column had missing values. Instead of removing all the rows with missing ages, I used the median age to fill in the missing values. I learned that the median can be useful because it is less affected by very high or very low values. Another problem was that columns such as Sex and Embarked contained categories instead of numbers. I used One-Hot Encoding to convert these categories into numerical values. Finally, I used feature scaling to put numerical features such as Age and Fare on a similar scale. Solving these problems step by step made the dataset easier for a machine learning model to process.

Real-World Applications

I can connect this lab to real-world data, especially data used in security and technology. For example, security systems can collect information from badge readers, cameras, sensors, and access-control systems. Sometimes information may be missing because a device stops working or does not record an event correctly. Data preparation techniques can help organize this information before it is analyzed. One-Hot Encoding can also be used for categories such as alarm types or locations. Feature scaling can be useful when different measurements have very different numerical ranges. This showed me that the ideas from this lab can be useful outside of the classroom.

Connecting New & Prior Knowledge

This lab connected with what I already knew about basic statistics. Before this lab, I knew that the median represents the middle value of a group of numbers, but I did not fully understand how it could be used in machine learning. Now I understand that statistical ideas can help solve practical data problems. I also learned that different machine learning models have different requirements. For example, models that depend on distances can be affected by the scale of the features, while a Decision Tree can often work without feature scaling. This helped me understand that data preparation should depend on the type of model being used.

Emerging Questions

One question I still have is how to choose the best method for missing data in a real situation. Using the median works well for some numerical data, but it may not always be the best choice. For example, if data is missing because of a technical problem or a security incident, replacing it with a typical value could hide an important pattern. I would like to learn more about how machine learning professionals decide whether to impute, remove, or investigate missing data.

Changing Approach & Critical Thinking

After completing this lab, I will look at machine learning more as a process of careful data preparation instead of only focusing on the model itself. I learned that a model can only work well when the data is prepared correctly. I also understand that I need to check my code and results carefully instead of assuming that the

output is correct. Overall, Lab 05 gave me a better understanding of missing values, One-Hot Encoding, feature scaling, and the importance of preparing data before using it in machine learning